

Design Patterns – Smart Parking Lot

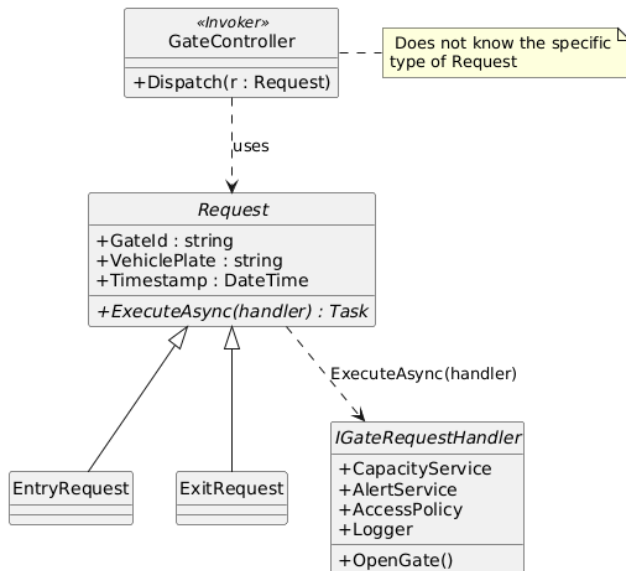
Command · Observer · Adapter · Strategy

Jesus G. Delgado. – Santiago Barrera M. – Johan S. Forero A. – David A. Monroy A.

1. Command (Behavioral)

Turns a request into a standalone object that encapsulates all the information needed to execute it. Decouples sender from receiver and allows requests to be queued, audited, or undone.

UML

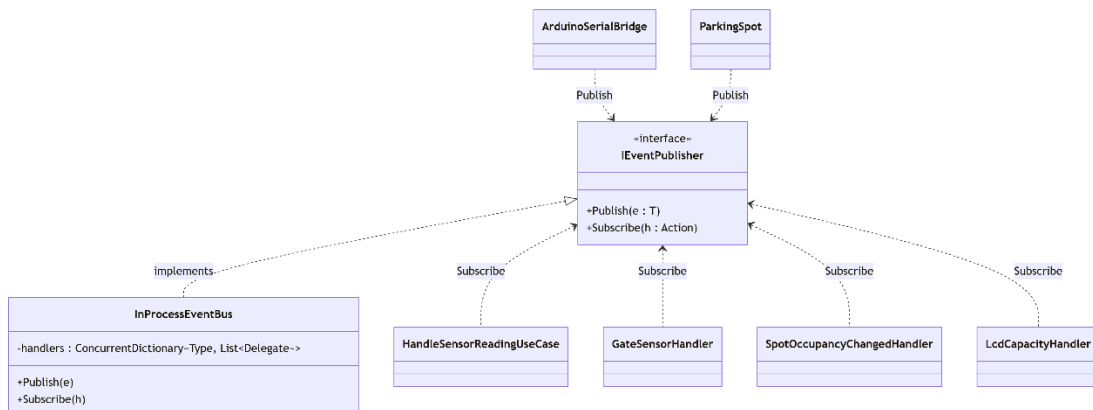


Every IR Sensor reading produces an **EntryRequest** or **ExitRequest**. The **GateController** (Invoker) only knows the base class and calls `request.ExecuteAsync(this)`. Adding a new request type does not require modifying the invoker. This materializes OCP (the invoker is open to new request types, closed to modification) and SRP (each Request owns one operation), and on the GRASP side it leverages Polymorphism and Protected Variations to shield **GateController** from change, keeping coupling between Invoker and Receiver low.

2. Observer (Behavioral)

Defines a one-to-many dependency: when a publisher changes state or emits an event, all of its subscribers are notified without any direct coupling.

UML



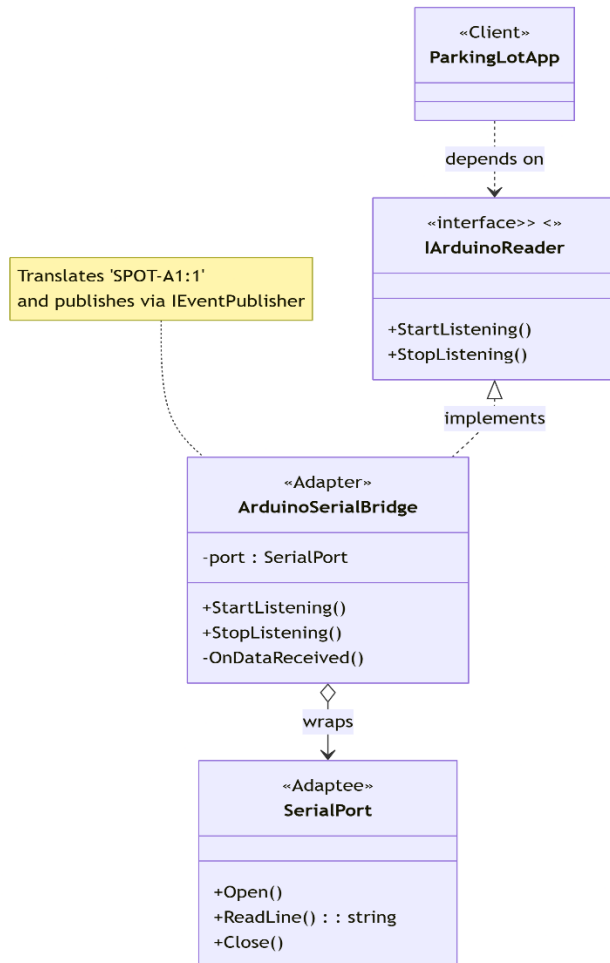
Application in the project

It is the central mechanism between hardware and application. A sensor reading (SensorReadingReceived) triggers several reactions in parallel: update occupancy, refresh the LCD, audit, and create the entry Request — all without the publisher being aware of them. The event bus acts as an Indirection (GRASP) that drives Low Coupling between publishers and subscribers and High Cohesion within each handler, while satisfying DIP (both sides depend on IEventPublisher, not on each other) and OCP (new subscribers can be added without touching the publisher).

3. Adapter (Structural)

Lets incompatible interfaces work together: it wraps an existing object and exposes the interface the client expects, without modifying either one.

UML



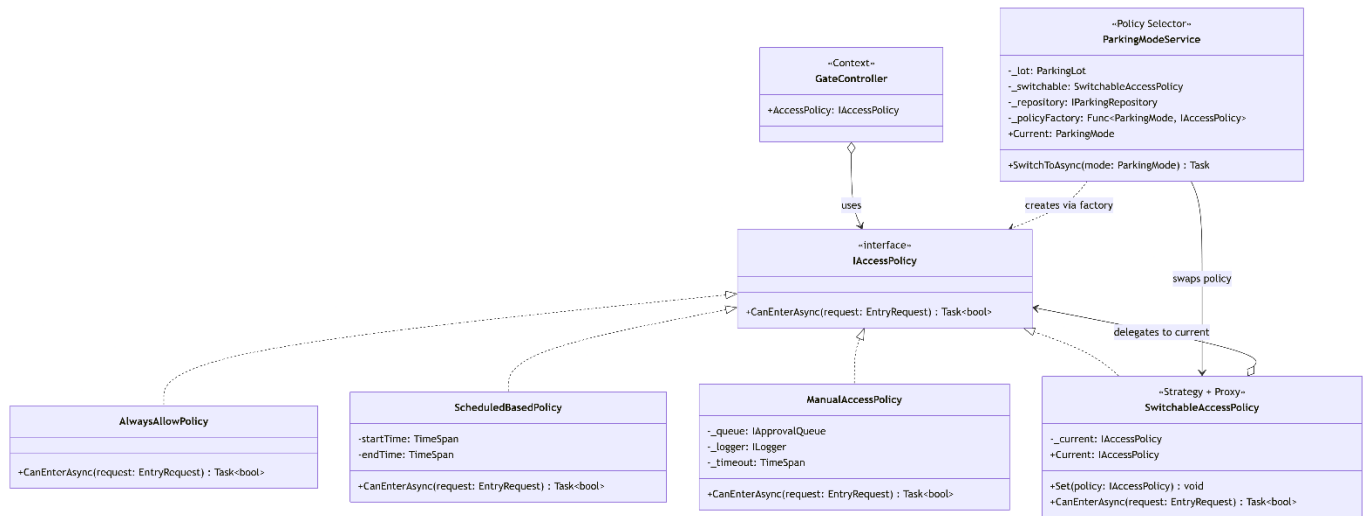
Application in the project

Isolates the domain from the hardware. `ArduinoSerialBridge` translates `SerialPort.Open/ReadLine` (`System.IO.Ports`) into the domain operations `StartListening / StopListening`, parses the serial protocol ("SPOT-A1:1"), and publishes events via `IEventPublisher`. In tests it is replaced with a mock without touching the domain. The adapter is a textbook Protected Variations (GRASP): the unstable hardware boundary is encapsulated behind `IArduinoReader`, which also enforces DIP (the domain depends on the abstraction, not on `System.IO.Ports`) and SRP (serial parsing lives in exactly one class), reducing coupling between domain and infrastructure.

4. Strategy (Behavioral)

Encapsulates a family of interchangeable algorithms behind a common abstraction. The client does not know which one it uses; it only knows the contract.

UML



Application in the project

Sometimes any vehicle may enter, sometimes a schedule applies, sometimes a whitelist, and sometimes the decision must wait for a human operator. Embedding that in `GateController` would mean a switch-over-enum that mixes coordination with decision and forces recompilation on every new variant — breaking SRP, OCP and LSP at once. Strategy promotes each algorithm to its own class behind a shared contract: `IAccessPolicy.CanEnterAsync(EntryRequest)`, implemented by `AlwaysAllowPolicy`, `ScheduledBasedPolicy`, and `ManualAccessPolicy`. `GateController` only delegates. To switch policies at runtime, `GateController` receives a `SwitchableAccessPolicy` (also `IAccessPolicy`) that internally points to the active one; `ParkingModeService` rebuilds that target via a factory (`Func<ParkingMode, IAccessPolicy>`) when the operator changes mode. Adding a policy means one new class plus a factory entry — no change to the controller or the entry flow — and each policy is unit-tested in isolation against an `EntryRequest`.

Why not an if/else with an enum

- Every new policy would force a change to `GateController` (breaks Open/Closed).
- Each policy's logic gets tangled with the entry flow (low cohesion).
- Impossible to switch modes at runtime (always-open / scheduled / manual / restricted) or to test each policy in isolation.

6. Comparison table

Pattern	Category	Intent	Role in the project
Command	Behavioral	Encapsulate a request as an object.	<code>EntryRequest/ExitRequest</code> executed by <code>GateController</code> .
Observer	Behavioral	Notify 1-to-N without coupling publisher and subscribers.	<code>InProcessEventBus</code> dispatches events from the Arduino.
Adapter	Structural	Adapt an existing interface to the one the client expects.	<code>ArduinoSerialBridge</code> wraps <code>SerialPort</code> behind <code>IArduinoReader</code> .
Strategy	Behavioral	Swap algorithms at runtime behind a common abstraction.	<code>IAccessPolicy</code> : <code>AlwaysAllow</code> / <code>ScheduledBased</code> / <code>Manual</code> , swapped at runtime via <code>SwitchableAccessPolicy</code> + <code>ParkingModeService</code> .